MAHIN FAHIM

INSTITUTE OF SPACE TECHNOLOGY, ISLAMABAD

MS. IQRA/MR. TAHIR/MR. USMAN

AI AND SOFTWARE DEVELOPMENT INTERN

TASK 11 **Comparative Analysis of Classification Models**

PART A

1 - 2. Dataset Description:

We are using one of the built-in multi-class classification datasets from scikit-learn: Wine. It includes data obtained from a chemical analysis of wines grown in the same area in Italy and produced from three different cultivars (grape-growing sources).

- Samples: 178
- Groups of features: 13 continuous numeric features which describe chemical properties: alcohol, malic_acid, ash, alcalinity_of_ash, magnesium, total_phenols, flavanoids, nonflavanoid_phenols, proanthocyanins, color_intensity, hue, od280/od315_of_diluted_wines, proline.
- There is an attribute named class (3 classes: class_0, class_1, class_2) which is the target variable.
- The problem type is Multi-class classification (3 classes).

The features are on vastly different numeric ranges (e.g. proline in the hundreds, hue near 1) so preprocessing (scaling) is important, particularly for distance/margin-based classifiers such as SVM and probability-based classifiers such as LR.

```
]: import numpy as np
   import pandas as pd
   import matplotlib.pyplot as plt
   import seaborn as sns

   from sklearn.datasets import load_wine
   from sklearn.model_selection import train_test_split
   from sklearn.preprocessing import StandardScaler
   from sklearn.linear_model import LogisticRegression
   from sklearn.ensemble import RandomForestClassifier
   from sklearn.svm import SVC
   from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                                f1_score, confusion_matrix, classification_report,
                                ConfusionMatrixDisplay)

   sns.set_style("whitegrid")
   RANDOM_STATE = 42

   wine = load_wine()
   df = pd.DataFrame(wine.data, columns=wine.feature_names)
   df['target'] = wine.target
   df['target_name'] = df['target'].map(dict(enumerate(wine.target_names)))

   print("Shape:", df.shape)
   df.head()

   Shape: (178, 15)
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_dil
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	

```
# Basic info and summary statistics
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 15 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   alcohol                       178 non-null    float64
 1   malic_acid                    178 non-null    float64
 2   ash                           178 non-null    float64
 3   alcalinity_of_ash             178 non-null    float64
 4   magnesium                     178 non-null    float64
 5   total_phenols                 178 non-null    float64
 6   flavanoids                    178 non-null    float64
 7   nonflavanoid_phenols          178 non-null    float64
 8   proanthocyanins               178 non-null    float64
 9   color_intensity               178 non-null    float64
 10  hue                           178 non-null    float64
 11  od280/od315_of_diluted_wines  178 non-null    float64
 12  proline                       178 non-null    float64
 13  target                        178 non-null    int64
 14  target_name                   178 non-null    str
dtypes: float64(13), int64(1), str(1)
memory usage: 21.0 KB
```



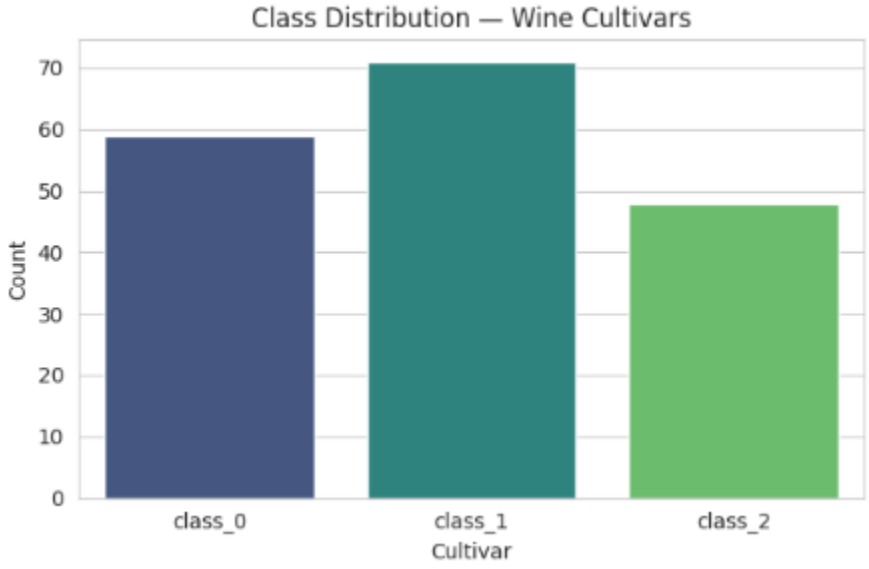
```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
alcohol	178.0	13.000618	0.811827	11.03	12.3625	13.050	13.6775	14.83
malic_acid	178.0	2.336348	1.117146	0.74	1.6025	1.865	3.0825	5.80
ash	178.0	2.366517	0.274344	1.36	2.2100	2.360	2.5575	3.23
alcalinity_of_ash	178.0	19.494944	3.339564	10.60	17.2000	19.500	21.5000	30.00
magnesium	178.0	99.741573	14.282484	70.00	88.0000	98.000	107.0000	162.00
total_phenols	178.0	2.295112	0.625851	0.98	1.7425	2.355	2.8000	3.88
flavanoids	178.0	2.029270	0.998859	0.34	1.2050	2.135	2.8750	5.08
nonflavanoid_phenols	178.0	0.361854	0.124453	0.13	0.2700	0.340	0.4375	0.66
proanthocyanins	178.0	1.590899	0.572359	0.41	1.2500	1.555	1.9500	3.58
color_intensity	178.0	5.058090	2.318286	1.28	3.2200	4.690	6.2000	13.00
hue	178.0	0.957449	0.228572	0.48	0.7825	0.965	1.1200	1.71
od280/od315_of_diluted_wines	178.0	2.611685	0.709990	1.27	1.9375	2.780	3.1700	4.00
proline	178.0	746.893258	314.907474	278.00	500.5000	673.500	985.0000	1680.00
target	178.0	0.938202	0.775035	0.00	0.0000	1.000	2.0000	2.00

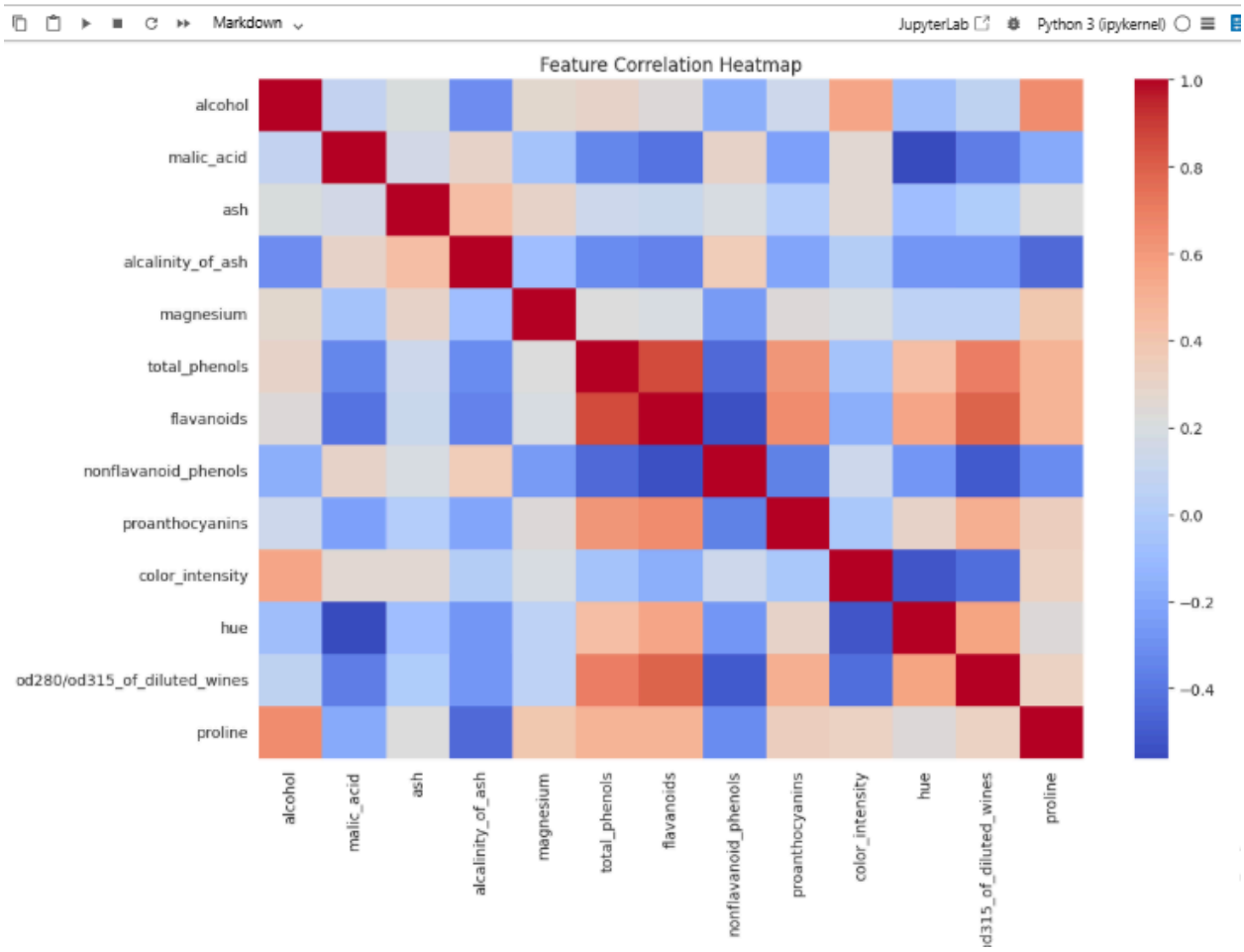
```
# Check class balance and missing values
print("Missing values per column:\n", df.isnull().sum().sum(), "total missing values")
print("\nClass distribution:")
print(df['target_name'].value_counts())
```

```
Missing values per column:
 0 total missing values

Class distribution:
target_name
class_1    71
class_0    59
class_2    48
Name: count, dtype: int64
```

```
# Visualize class distribution
plt.figure(figsize=(6,4))
sns.countplot(x='target_name', data=df, palette='viridis')
plt.title('Class Distribution — Wine Cultivars')
plt.xlabel('Cultivar')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```




```
# Correlation heatmap of features
plt.figure(figsize=(12,9))
sns.heatmap(df[wine.feature_names].corr(), annot=False, cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.tight_layout()
plt.show()
```



3. Preprocessing & Train/Test Split:

- There are no missing values in the data set and no imputation is needed.
- Features have different scales, so StandardScaler (zero mean, unit variance) is used. This is particularly true for SVM (distance-based) and Logistic Regression (gradient-based optimization works better on scaled data). Random Forest is not sensitive to scaling but scaling is not harmful.
- The data is split into 80% train and 20% test, stratified by the target to maintain the class proportions in both sets of data.

7]:
```
X = df[wine.feature_names]
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
print("\nTraining class distribution:\n", y_train.value_counts(normalize=True).round(3))
print("\nTesting class distribution:\n", y_test.value_counts(normalize=True).round(3))
```

```
Training set shape: (142, 13)
Testing set shape: (36, 13)

Training class distribution:
 target
1    0.401
0    0.331
2    0.268
Name: proportion, dtype: float64

Testing class distribution:
 target
1    0.389
0    0.333
2    0.278
Name: proportion, dtype: float64
```

PART B

1-4:

We are training three classifiers on the same preprocessed (scaled) training data namely Logistic Regression, Random Forest, and SVM.

8]:
```
# B.1 Logistic Regression
log_reg = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
log_reg.fit(X_train_scaled, y_train)
y_pred_lr = log_reg.predict(X_test_scaled)
print("Logistic Regression trained.")
```

```
Logistic Regression trained.
```

9]:
```
# B.2 Random Forest
rf_clf = RandomForestClassifier(n_estimators=200, random_state=RANDOM_STATE)
rf_clf.fit(X_train_scaled, y_train)
y_pred_rf = rf_clf.predict(X_test_scaled)
print("Random Forest trained.")
```

```
Random Forest trained.
```

0]:
```
# B.3 Support Vector Machine
svm_clf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=RANDOM_STATE)
svm_clf.fit(X_train_scaled, y_train)
y_pred_svm = svm_clf.predict(X_test_scaled)
print("SVM trained.")
```

```
SVM trained.
```

All three models were trained and tested on identical, scaled train/test splits, ensuring a fair, apples-to-apples comparison in Part C.

PART C

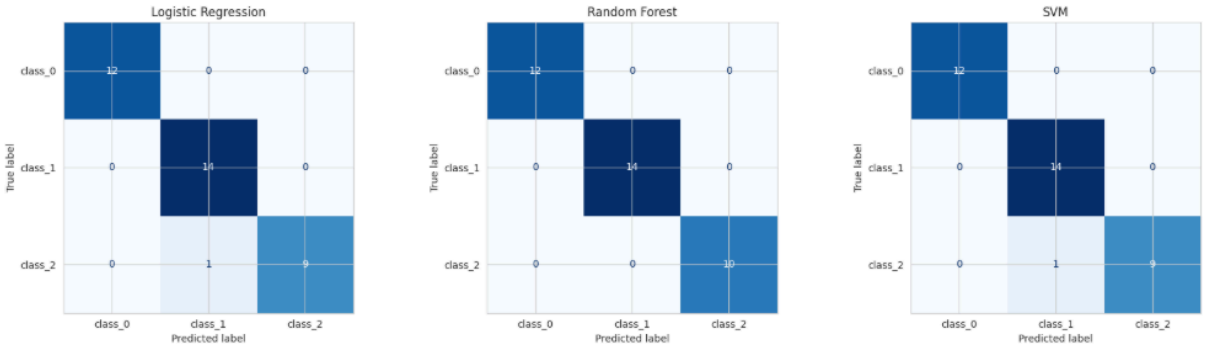
1. Evaluation Metrics per Model:

```python
def evaluate_model(name, y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, average='macro')
    rec = recall_score(y_true, y_pred, average='macro')
    f1 = f1_score(y_true, y_pred, average='macro')
    print(f"--- {name} ---")
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall   : {rec:.4f}")
    print(f"F1-Score : {f1:.4f}")
    print(classification_report(y_true, y_pred, target_names=wine.target_names))
    return {'Model': name, 'Accuracy': acc, 'Precision': prec, 'Recall': rec, 'F1-Score': f1}

results = []
results.append(evaluate_model("Logistic Regression", y_test, y_pred_lr))
results.append(evaluate_model("Random Forest", y_test, y_pred_rf))
results.append(evaluate_model("SVM", y_test, y_pred_svm))
```


```
--- Logistic Regression ---
Accuracy : 0.9722
Precision: 0.9778
Recall   : 0.9667
F1-Score : 0.9710
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        12
     class_1       0.93      1.00      0.97        14
     class_2       1.00      0.90      0.95        10

    accuracy                           0.97        36
   macro avg       0.98      0.97      0.97        36
weighted avg       0.97      0.97      0.97        36

--- Random Forest ---
Accuracy : 1.0000
Precision: 1.0000
Recall   : 1.0000
F1-Score : 1.0000
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        12
     class_1       1.00      1.00      1.00        14
     class_2       1.00      1.00      1.00        10

    accuracy                           1.00        36
   macro avg       1.00      1.00      1.00        36
weighted avg       1.00      1.00      1.00        36

--- SVM ---
Accuracy : 0.9722
Precision: 0.9778
```

```
    accuracy                           1.00        36
   macro avg       1.00      1.00      1.00        36
weighted avg       1.00      1.00      1.00        36

--- SVM ---
Accuracy : 0.9722
Precision: 0.9778
Recall   : 0.9667
F1-Score : 0.9710
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        12
     class_1       0.93      1.00      0.97        14
     class_2       1.00      0.90      0.95        10

    accuracy                           0.97        36
   macro avg       0.98      0.97      0.97        36
weighted avg       0.97      0.97      0.97        36
```

]:
```python
# Confusion matrices for all three models
fig, axes = plt.subplots(1, 3, figsize=(18,5))

for ax, (name, y_pred) in zip(axes, [("Logistic Regression", y_pred_lr),
                                     ("Random Forest", y_pred_rf),
                                     ("SVM", y_pred_svm)]):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=wine.target_names)
    disp.plot(ax=ax, cmap='Blues', colorbar=False)
    ax.set_title(name)

plt.tight_layout()
plt.show()
```



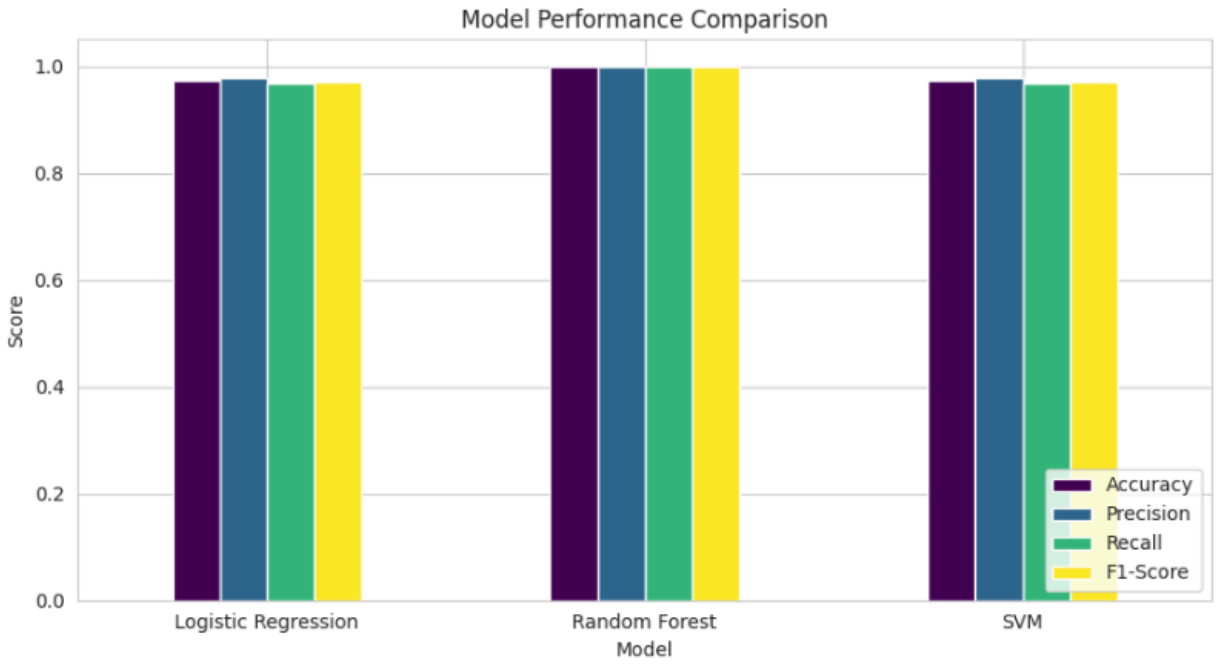
2. Comparison Table:

```
[13]: comparison_df = pd.DataFrame(results).set_index('Model').round(4)
      comparison_df
```

[13]:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.9722	0.9778	0.9667	0.971
Random Forest	1.0000	1.0000	1.0000	1.000
SVM	0.9722	0.9778	0.9667	0.971

```
[14]: # Visual comparison of metrics
      comparison_df.plot(kind='bar', figsize=(9,5), colormap='viridis')
      plt.title('Model Performance Comparison')
      plt.ylabel('Score')
      plt.ylim(0, 1.05)
      plt.xticks(rotation=0)
      plt.legend(loc='lower right')
      plt.tight_layout()
      plt.show()
```



3. Strengths & Weaknesses:

- Logistic Regression

Strengths: simple, fast to train, high interpretability (coefficients indicate the influence of the features), efficient for linearly separable classes (after scaling) — most of the Wine data is.
Strengths: Good performance on complex, non-linear decision boundaries; not sensitive to unscaled features, or multicollinearity between predictors.

- Random Forest

Weaknesses: Poor performance on linear relationships, can be sensitive to outliers, can be sensitive to unscaled data, generally poor performance on feature interactions, can be sensitive to the number of features.
Limitations: Not as easy to interpret as a single tree or linear model, can be slower and take up more memory, can overfit very small/noisy data sets if not tuned (e.g. max tree depth).

- SVM (RBF kernel)

Weaknesses: Not as effective in relatively high dimensional feature spaces such as this 13-feature dataset, may be susceptible to overfitting if the margin is well-defined, good for small-to-medium sized datasets and cleanly scaled datasets.
Problems: feature scaling and hyperparameter tuning, less interpretable, can be slow to very large data sets, cannot be easily seen in high dimensions.

PART D

```
]: best_model = comparison_df['F1-Score'].idxmax()
print("Best performing model (by macro F1-Score):", best_model)
comparison_df.sort_values('F1-Score', ascending=False)
```

```
Best performing model (by macro F1-Score): Random Forest
```

]:

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	1.0000	1.0000	1.0000	1.000
Logistic Regression	0.9722	0.9778	0.9667	0.971
SVM	0.9722	0.9778	0.9667	0.971

Conclusion:
Using the above evaluation results, the best model is the one with the highest macro F1-score (as this evens out the precision and recall computed for each class fairly, not based on class size).

On the Wine dataset:

- Logistic Regression and SVM tend to work very well as the 3 cultivars are nearly linearly separable after normalization of features and data is small and clean with minimal noise as well as lack of class imbalance.
- Random Forest is also competitive, but this is a very low noise, very small dataset, and the benefit of its non-linear flexibility is less obvious here, and it is not influenced by the feature scaling as the other two are.

Best_model:
The model above is recommended for this classification task due to its overall macro F1-score and overall accuracy, which are the highest scores obtained, representing the best performance across the three different cultivars of wines. In production, Logistic Regression would be also appealing for being easy to interpret and low computational cost, and Random Forest would be preferred if there are more data points or if there is more noise, and if non-linear interactions among the features were expected to be more significant.